Sessione

- Una sessione è un'interazione a lungo termine fra un client e un server. Può consistere di una singola richiesta o di una serie di richieste. Tipicamente una sessione inizia quando un utente fa login, e termina quando l'utente fa log out o semplicemente va via
- Sessioni (o server) stateful e stateless
  - Con server stateless si intende un server che non mantiene (conserva) uno stato fra le richieste
    - Durante l'interazione con l'oggetto server, vari campi dell'oggetto prenderanno certi valori, quindi l'oggetto ha uno stato
    - Quando la risposta è stata data, lo stato dell'oggetto è inutile. Ad una nuova richiesta lo stato viene reimpostato
  - Un server stateful tiene traccia di ciò che il client da un certo indirizzo IP ha visitato. La lista di pagine visitate è mantenuta fra una richiesta e l'altra

Stato Del Server

- Un server stateful usa più risorse di un server stateless
- In un server stateful dovremo usare lo stesso oggetto per servire le richieste dello stesso utente
  - Durante l'attesa di richieste, l'oggetto server deve tenere il suo stato. Per ciascun utente vi sarà un corrispondente oggetto server, es. per 100 utenti, vi saranno 100 oggetti server
  - Se l'utente interagisce ogni 10 secondi e la computazione lato server prende un secondo, l'oggetto server aspetta in memoria per il 90% del tempo
- In un server stateless, non importa quale oggetto usiamo per servire una richiesta

Stato Della Sessione

- Un server che ha tanti utenti dovrebbe essere stateless, per evitare un enorme uso di risorse
- D'altra parte, per fornire una funzionalità tipo carrello della spesa la sessione dovrebbe essere stateful
- I dettagli del carrello della spesa sono lo stato della sessione, ovvero sono rilevanti solo per quella sessione
- I dati di sessione sono distinti dai dati permanenti che devono essere conservati e disponibili per tutte le sessioni
  - Alcuni dati di sessione possono essere conservati per diventare dati permanenti
- Ci sono tre scelte di base per conservare lo stato della sessione: sul client, sul server, sul database

Design Pattern

Client Session State

Client Session State

- Intento: conserva lo stato della sessione sul client
- Come funziona
  - Tutti i dati della sessione sono sul client, quindi il client manda tutti i dati di sessione con ogni richiesta al server, e il server manda tutto lo stato con ogni risposta
  - Si può usare un *Data Transfer Object* per tenere i dati
  - Con un rich-client, i dati possono essere tenuti da campi dell'interfaccia, o (meglio) nello stesso Data Transfer Object senza essere visualizzati
  - Con un'interfaccia HTML ci sono tre modi: parametri dell'URL, campi nascosti, cookie

Client Session State

- Dati di sessione su URL
  - La dimensione dell'URL è limitata
  - Con pochi dati, per es. un ID di sessione, funziona bene e può essere una scelta ricorrente
  - L'associazione di un ID di sessione può essere fatta in automatico dalla piattaforma cambiando l'URL, quindi il bookmark può essere un problema
- Dati di sessione su campi nascosti
  - Un campo nascosto è un campo inviato al browser ma non visualizzato sulla pagina web. Tag: `<input type="hidden">`
  - Lo stato della sessione è inserito in un campo nascosto e serializzato con la risposta. XML può essere usato come formato per i dati inseriti nel campo nascosto

Client Session State

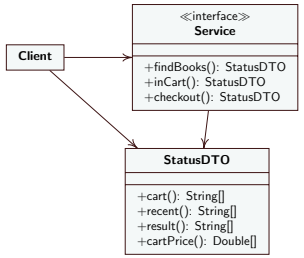
- Dati di sessione su cookie
  - I cookie sono inviati avanti e indietro automaticamente
  - Lo stato della sessione è serializzato su un cookie
  - Se l'utente disattiva i cookie per il suo browser il sito non funziona
  - Un cookie funziona solo per un singolo nome di dominio, quindi siti separati in diversi nomi di dominio non vedranno i cookie di altri siti

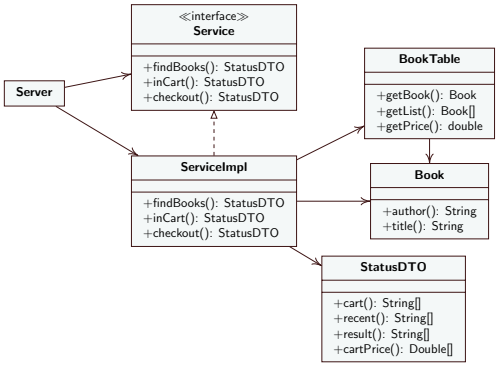
Client Session State

- Quando usarlo
  - Funziona bene insieme a server object stateless, e riesce a far recuperare la sessione quando un server va giù e viene sostituito
  - Quando il client va giù si perde tutto, ma l'utente se lo aspetta
  - Si usa quasi sempre per identificare la sessione. Per evitare che qualcuno catturi una sessione di un altro (cambiando l'ID di sessione) si usa l'hash di un ID semplice
- Possibili svantaggi
  - Con grandi quantità di dati, il tempo di trasferimento durante una richiesta può diventare proibitivo
  - I dati inviati al client possono essere visti e alterati, i dati re-inviati al server devono essere ri-convalidati. Il criptaggio e decriptaggio ad ogni richiesta può essere un problema per le prestazioni

Esempio

- Lato client: `Client`, `Service`, `StatusDTO`
- Lato server: `Service`, `ServiceImpl`, `BookTable`, `Book`, `StatusDTO`
- `StatusDTO` è un DTO (stato: `cart`, `recent`, `result`, `cartPrice`)
- `Service` è una interfaccia per le chiamate RMI





Design Pattern

Server Session State

Server Session State

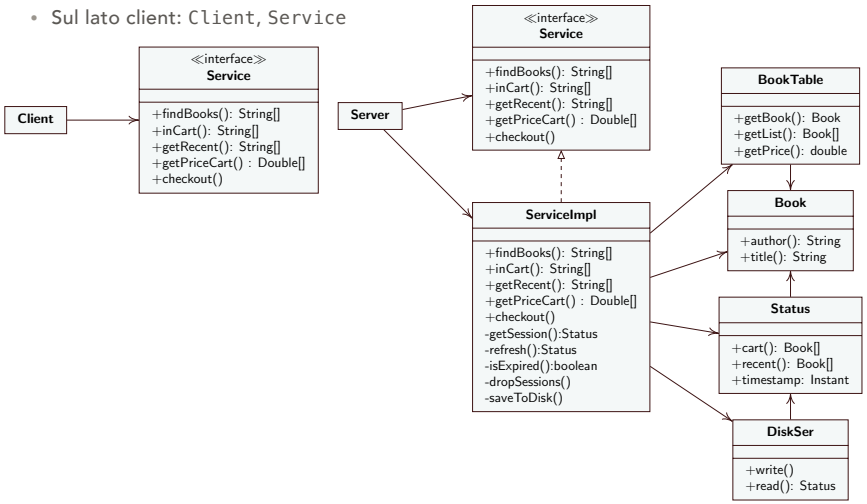
- Intento: conserva lo stato della sessione sul server in forma serializzata
- Come funziona
  - Un oggetto di sessione è tenuto in memoria sul lato server dell'applicazione
  - Una mappa in memoria tiene il riferimento a oggetti di sessione tramite la chiave pari all'ID di sessione. Dall'ID si prende l'oggetto dalla mappa e si può servire la richiesta del client
  - Si ha: (i) sufficiente memoria sul server, (ii) un solo application server
  - Per risolvere il problema della memoria limitata, si serializza lo stato dell'oggetto sessione (vedi Memento [GoF]) e si immagazzina su uno storage, ovvero l'oggetto di sessione è *passivato*
    - Si può serializzare in binario o in XML

Server Session State

- Lo storage può essere il file system o un database dello stesso host
- Se lo storage è accessibile da più host, si rende possibile
  - Clustering, ovvero distribuzione di richieste a più server
  - Failover, se un host va giù, il controllo si trasferisce ad un altro host che recupera lo stato
- Si può aver bisogno del pattern *Serialized LOB* per tenere lo stato
- Se si memorizza lo stato in un database, ci si deve occupare della gestione delle sessioni che vanno via, le vecchie sessioni devono essere cancellate controllando periodicamente i timestamp sul database
- Quando usarlo
  - Il punto di forza è la semplicità, se vi è sufficiente memoria, tuttavia clustering e failover possono essere complicati da gestire

Esempio

- Sul lato server: `Server`, `Service`, `ServiceImpl`, `BookTable`, `Book`, `Status`, `DiskSer`
- Sul lato client: `Client`, `Service`



Design Pattern

Database Session State

Database Session State

- Intento: conserva dati di sessione come dati dedicati su database
- Come funziona
  - Quando una richiesta del client arriva sul server, anzitutto il server prende i dati necessari dal database
  - Per riprendere i dati della sessione è necessario almeno un ID di sessione che è conservato sul client (è la chiave per l'accesso ai dati del database)
  - I dati recuperati sono tipicamente dati di sessione (utili solo per l'interazione corrente) e dati rilevanti per tutte le interazioni
  - <u>I dati di sessione devono essere considerati locali</u>, e altre parti del sistema non dovrebbero usarli, fino a quando non si fa commit della sessione

Database Session State

- I dati di sessione conservati sul database devono essere considerati dati intermedi e distinti dagli altri che sono stati confermati
  - Aggiungere un campo nella riga del database che indica che i dati della riga sono in sospeso (pending)
    - Campo `isPending` di tipo `boolean`, oppure
    - Campo `isPending` che conserva l'ID di sessione, così che sarà facile trovare tutti i dati per una certa sessione
    - Aggiungere questo campo costringe altre parti (altre applicazioni) a conoscerne il significato e gestirlo (invasività)
  - Inserire un set di tabelle separate solo per dati di sessione (pending)
    - Quando i dati diventano permanenti saranno conservati in altre tabelle
    - Rimuove invasività, il codice deve mappare sulle tabelle opportune

Database Session State

- Tipicamente ci sono regole per la convalida dei dati conservati sul database che assicurano l'integrità dei dati. Tali regole non vanno applicate ai dati pending, che invece potrebbero necessitare di altre regole. Avere tabelle separate facilita l'applicazione di regole diverse
- Le tabelle di dati pending dovrebbero essere clonate dalle tabelle reali, ovvero stessi nomi di campi fra le due tabelle, e un campo aggiuntivo per le tabelle pending per l'ID di sessione
  - Questo facilita l'implementazione della logica di mapping
- Se una sessione è cancellata l'ID di sessione permette di trovare tutti i dati da cancellare, oppure
- Se la sessione è abbandonata, si può usare un timeout, un daemon esegue ad intervalli di pochi minuti per trovare e cancellare i vecchi dati
  - La tabella del database deve tener traccia del momento dell'ultima interazione con la sessione

Database Session State

- Quando usarlo
  - Permette di avere server con object che non tengono lo stato, quindi abilita il pooling e facilita il clustering
  - E' necessario del tempo per l'accesso ai dati per ciascuna richiesta. Una cache può ridurre questi tempi
  - La programmazione della gestione dello stato della sessione può essere semplificata non tenendo lo stato in memoria, ma conservandolo sul database
  - Clustering e failover sono più facili con il Database Session State

Design Pattern

Serialized LOB

Serialized LOB

- Intento: conserva un grafo di oggetti serializzandoli dentro un singolo grande oggetto (LOB), che è inserito in un campo di un database
- Problema
  - I sistemi ad oggetti consistono di grafi complessi di piccoli oggetti. Parte dell'informazione non è negli oggetti ma nelle loro interconnessioni
  - La gerarchia dell'organizzazione per acquirenti di prodotti può essere modellata facilmente da composizioni di oggetti con metodi che permettono di trovare oggetti in base alle relazioni (es. oggetti fratelli nella gerarchia)
  - Non è banale mettere queste informazioni in uno schema relazionale. Lo schema base è semplice: una tabella con una chiave esterna, ma manipolare tale schema richiede molti join che rallentano e sono poco maneggevoli
  - Gli oggetti non devono necessariamente essere conservati su righe di tabelle con relazioni. Un'altra forma di persistenza è la serializzazione, in cui tutto il grafo di oggetti è scritto come un singolo grande oggetto (LOB) in una tabella

Serialized LOB

- Come funziona
  - La serializzazione può essere fatta in binario (BLOB) o come caratteri (CLOB)
  - Un oggetto serializzato è un oggetto rappresentato come una sequenza di byte che include i dati dell'oggetto e informazioni sul tipo dell'oggetto e sui tipi di dati conservati nell'oggetto
  - Quando si usa il Serialized LOB bisogna fare attenzione ai problemi di duplicazione di dati
    - Un Serialized LOB per i dettagli di un cliente potrebbero essere messi su un ordine, se il LOB si inserisce sulla tabella degli ordini i dati del cliente verranno ricopiati per ciascun ordine
    - Il LOB può essere messo in una tabella dei clienti e gli ordini possono avere un link ad essa
  - In generale, una parte dei dati del LOB potrebbe venir duplicata

Serializable

- Avvertenze
  - Se nel grafo da memorizzare, a partire dall'oggetto passato a `writeObject()` di `ObjectOutputStream`, si incontrasse un riferimento a un oggetto la cui classe non implementa `Serializable` si avrebbe un errore a runtime
  - Se si vuol escludere dalla memorizzazione un campo di un oggetto del grafo, il campo dovrà essere dichiarato `transient`
  - Se un campo che è il riferimento a un oggetto è dichiarato `transient`, la corrispondente classe non necessita di essere `Serializable`